Laboratory 1: Building a Simple HTTP Client Using Java Sockets

```
import java.net.Socket;
import java.io.*;

public class lab2 {
    public static void main(String[] args) throws IOException {
        String host = "example.com";
        int port = 80;
        Socket socket = new Socket(host, port);
        OutputStream out = socket.getOutputStream();
        BufferedReader reader = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));

        String request =
                "GET / HTTP/1.1\r\n" +
                        "Host: example.com\r\n" +
                        "Connection: close\r\n" +
                        "\r\n";

        out.write(request.getBytes());
        out.flush();

        String line;
        while ((line = reader.readLine()) != null) {
            System.out.println(line);
        }

        socket.close();
    }
}
```

Laboratory 2: Echo Client–Server Communication Using Java Sockets

Echo Server:

```
import java.io.*;
import java.net.*;
import java.util.*;

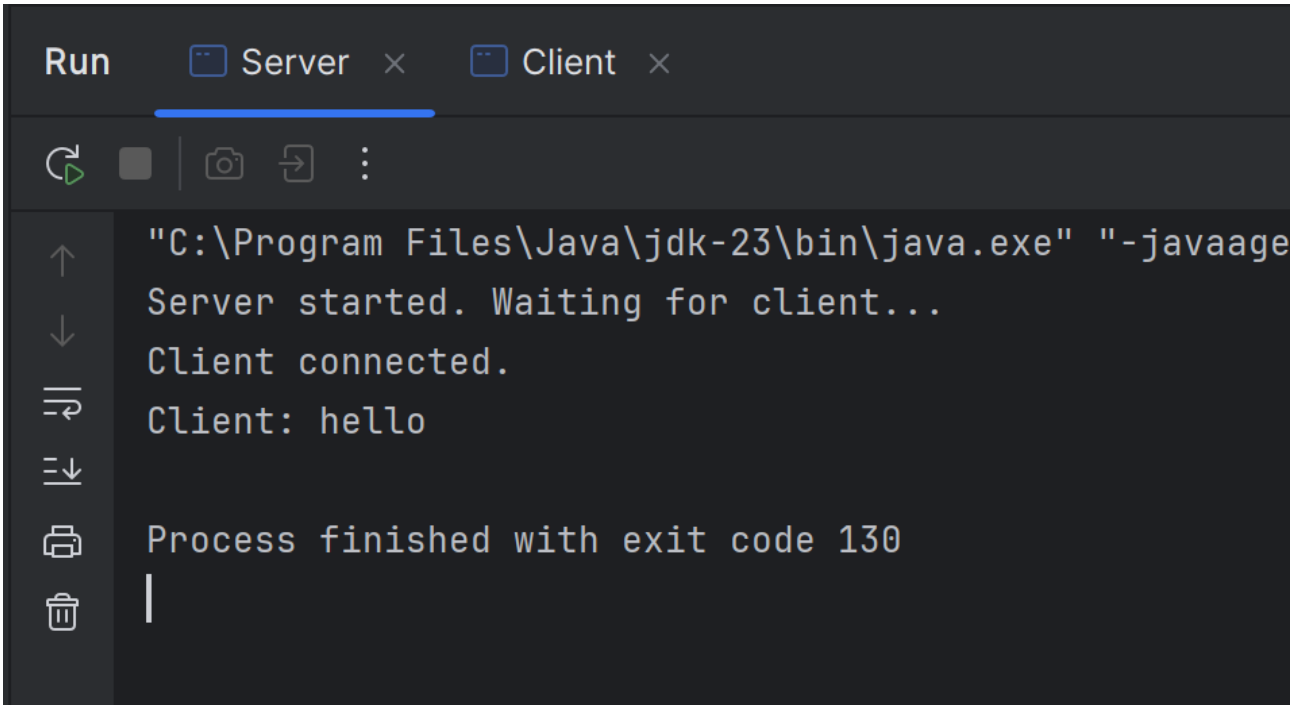
public class echoServer {
    public static void main(String[] args) {
        try {
            ServerSocket server = new ServerSocket(5000);
            System.out.println("Server started. Waiting for client...");
            Socket client = server.accept();
            System.out.println("Client connected.");

            BufferedReader in = new BufferedReader(
                    new InputStreamReader(client.getInputStream()));
            PrintWriter out = new PrintWriter(
                    client.getOutputStream(), true);
            String message;


            while ((message = in.readLine()) != null) {
                System.out.println("Client: " + message);
```

```
                out.println("Echo: " + message);
            }
            client.close();
            server.close();

        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```



Echo Client:

```
import java.io.*;
import java.net.*;
import java.util.*;

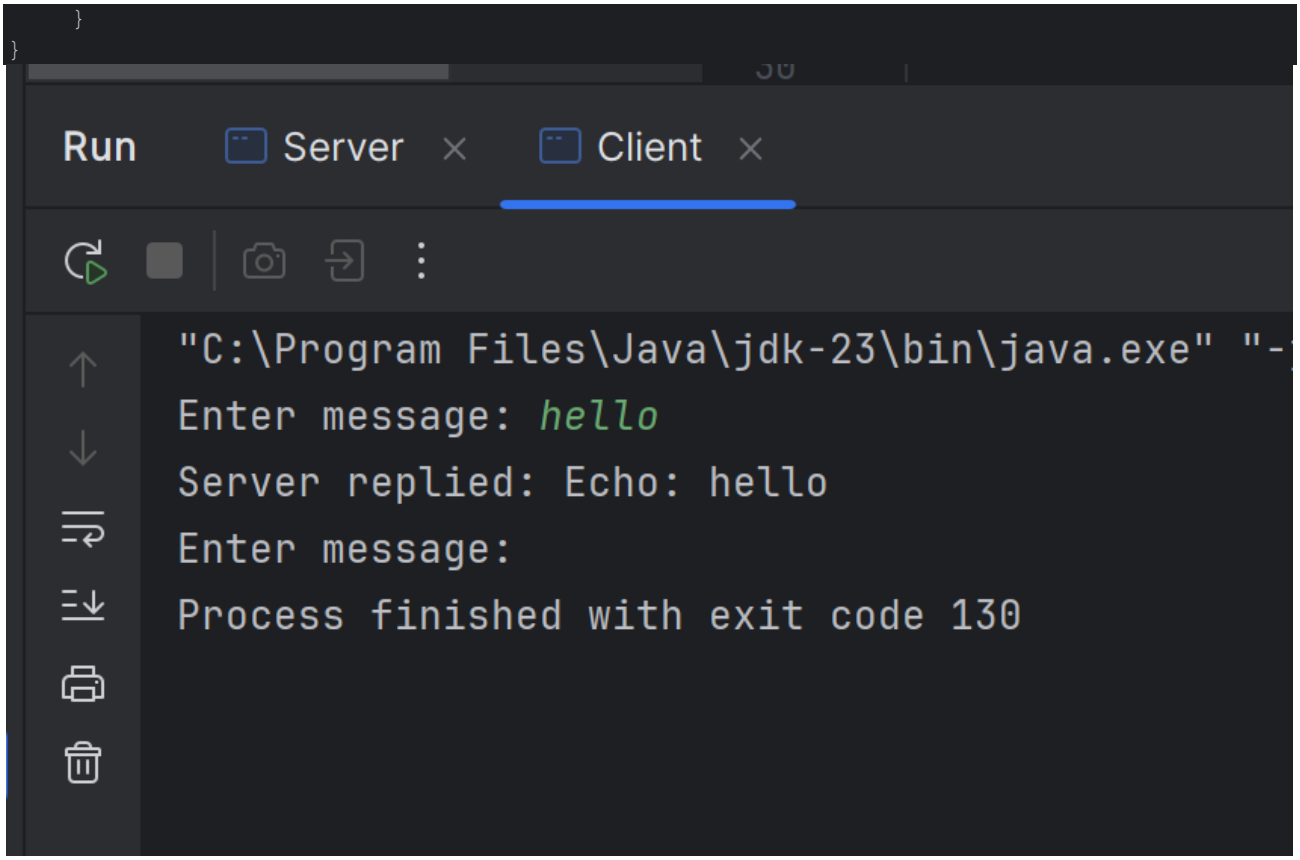
public class echoClient {
    public static void main(String[] args) {
        try {
            Socket socket = new Socket("localhost", 5000);
            BufferedReader in = new BufferedReader(
                    new InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(
                    socket.getOutputStream(), true);
            Scanner scanner = new Scanner(System.in);

            while (true) {

                System.out.print("Enter message: ");
                String messg = scanner.nextLine();
                out.println(messg);

                String response = in.readLine();
                System.out.println("Server replied: " + response);
            }

        } catch (IOException e) {
            e.printStackTrace();
        }
```



In this laboratory task I created a simple HTTP client using Java sockets. The program connected to a web server on port 80 and sent a manual HTTP GET request. After that, it received and printed the raw HTTP response in the console.

From the output I observed that an HTTP response has three main parts: the status line, the headers, and the message body.

The first line is the status line, for example HTTP/1.1 200 OK. It shows the HTTP version and the status code. The code 200 means that the request was successful.

The next part contains HTTP headers, which give additional information about the response, such as the type of content (Content-Type) or the size of the data (Content-Length).

After the headers there is an empty line, and then the message body, which contains the actual content returned by the server, usually HTML code of the webpage.